Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«ЮЖНЫЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»
(ФГАОУ ВО «ЮФУ»)
Институт компьютерных технологий и информационной безопасности
Информационная безопасность телекоммуникационных систем

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 3

по дисциплине:
«Машинное обучение и аналитика больших данных»

на тему:
«Высокопроизводительная геопространственная интерполяция метеорологических данных»

Выполнил: студент группы КТбо3-3 _____ Иршко О. А.

Выполнил: студент группы КТбо3-3 _____ Грицынин Н. С.

Проверил: Ассистент ИКТИБ ЮФУ _____ Мельник Я. Э.

Таганрог 2026

Оглавление

Цель работы 3
Краткие теоретические сведения 3
Ход работы 5
1. Постановка задачи 5
2. Модульная структура программного комплекса 5
3. Генерация входных данных (data_generator.py) 6
4. Векторизованное ядро интерполяции (interpolation_core.py) 6
5. Статистический анализ (stats_calculator.py) 7
6. Координация вычислений и сохранение в GeoTIFF (main.py) 7
7. Распределенные клиент-серверные вычисления (main_distributed.py) 8
Экспериментальное исследование и сравнительный анализ 9
1. Исследование зависимости времени интерполяции от числа процессов . 9
2. Визуализация результатов интерполяции 10
3. Апробация распределенных вычислений и верификация точности 10
Анализ результатов экспериментов 11
ВЫВОД 11

Цель работы

1. Освоить фундаментальные принципы и практические методы высокопроизводительной геопространственной интерполяции больших объемов данных на языке Python.

2. Научиться использовать специализированные геоинформационные библиотеки: расчет геодезических расстояний на сфере и эллипсоиде (pyproj), сохранение геопривязанных многомерных растров с компрессией (rasterio), организацию CLI-интерфейса (argparse).

3. Реализовать векторизованный алгоритм интерполяции методом обратно взвешенных расстояний (IDW) с пакетной обработкой строк (батчингом) для жесткого контроля оперативной памяти и потоковым вычислением временной статистики ячеек матрицы.

4. Разработать клиент-серверное расширение для распределенных вычислений на базе FastAPI в локальной сети, провести сравнительный анализ производительности и исследовать масштабируемость системы.

Краткие теоретические сведения

1. Метод IDW: Метод обратно взвешенных расстояний (Inverse Distance Weighting, IDW) является классическим алгоритмом пространственной интерполяции, основанным на предположении, что влияние известных точек уменьшается с увеличением расстояния до оцениваемой ячейки. Значение параметра Z в точке с координатами (x, y) вычисляется как средневзвешенное по формуле:

$Z(x, y) = [\sum(w_i \cdot z_i)] / [\sum w_i]$, где веса $w_i = 1 / (d_i^p + \varepsilon)$. Параметр p задает степень затухания расстояния (обычно p = 2.0), d_i — физическое расстояние от целевой точки до i-й опорной станции, а ε — малая константа (например, 10^{-12}), исключающая деление на ноль.

2. Геодезические расчеты на эллипсоиде WGS84: При работе в географической системе координат WGS84 (EPSG:4326) координаты задаются в градусах долготы и широты. Применение евклидова расстояния

на градусной сетке порождает грубые искажения, поскольку длина градуса долготы зависит от широты. Модуль pyproj.Geod(ellps='WGS84') решает обратную геодезическую задачу по алгоритму Винсенти, обеспечивая точный расчет физических расстояний в метрах по поверхности земного эллипсоида.

3. Векторизация в NumPy: Для обработки сотен тысяч пикселей за 365 временных срезов циклы Python неприменимы из-за критических накладных расходов. Векторизация средствами NumPy переносит расчеты в скомпилированные библиотеки BLAS/LAPACK. В частности, интерполяция показаний станций на 365 дней сводится к единой операции матричного перемножения нормализованных весов на матрицу дневных температур станций (оператор @).

4. Пакетная обработка (батчинг) и контроль RAM: Хранение полного массива размером 400 × 800 × 365 элементов в формате float32 требует порядка 467 МБ RAM. При высоком разрешении сетки потребление памяти может превысить доступный объем ОЗУ. Пакетная обработка (батчинг) делит растр на горизонтальные полосы строк. Для каждого батча рассчитываются значения сразу за весь год, сворачиваются в статистику (Min, Max, Mean) и удаляются из памяти, что снижает пиковое потребление RAM более чем в 10 раз.

5. Формат GeoTIFF и компрессия LZW: Библиотека rasterio обеспечивает запись растровых данных в формат GeoTIFF с указанием системы координат (EPSG:4326), аффинного преобразования (связь индексов матрицы с географическими координатами) и сжатия LZW без потерь, уменьшающего объем файлов в 2–3 раза.

6. Обработка пропущенных значений (NaN): Для агрегации временных рядов при наличии возможных пропусков данных методичка предписывает обязательное использование nan-безопасных функций NumPy (np.nanmin, np.nanmax, np.nanmean), предотвращающих каскадную порчу статистики.

Ход работы

1. Постановка задачи

В соответствии с Вариантом 1 («Температура воздуха») зафиксированы следующие условия:

– **Регион исследования:** Полуостров Крым (WGS84: долгота 32.4°E – 36.6°E, широта 44.3°N – 46.2°N).

– **Опорная сеть:** 50 станций, случайно расположенных в границах региона.

– **Параметр и диапазон:** температура воздуха в диапазоне от -50.0 °C до +50.0 °C на 365 дней.

– **Сетка расчета:** матрица 400 строк на 800 столбцов (320 000 ячеек растра).

– **Результирующие слои:** минимум, максимум, среднегодовое значение за 365 дней.

2. Модульная структура программного комплекса

Проект разделен на отдельные модули в соответствии с рекомендациями методички:

– **data_generator.py –** генерация синтетической базы станций и ежедневных наблюдений в CSV-файл.

– **interpolation_core.py –** векторизованный расчет геодезических расстояний через pyproj, матричные веса IDW и свертка временного ряда.

– **stats_calculator.py –** потоковый расчет предварительной статистики по CSV и агрегация батчей через nan-безопасные функции NumPy.

– **main.py –** интерфейс командной строки (argparse), диспетчер ProcessPoolExecutor, механизм батчинга и сохранение в GeoTIFF (rasterio).

– **main_distributed.py –** клиент-серверный вычислительный комплекс на базе FastAPI для параллельного расчета на двух ПК.

– **benchmark.py** – автоматизированное тестирование производительности от 1 до 12 процессов.

3. Генерация входных данных (data_generator.py)

Функция generate_data создает 50 станций со случайными координатами в границах Крыма и для каждой генерирует 365 суточных значений температуры:

```
def generate_data(filename="stations.csv", num_stations=50,
bounds=None):
    if bounds is None:
        bounds = {"west": 32.4, "east": 36.6, "south": 44.3, "north":
46.2}
    stations = {}
    for i in range(num_stations):
        lat = random.uniform(bounds["south"], bounds["north"])
        lon = random.uniform(bounds["west"], bounds["east"])
        stations[f"station_{i}"] = {"latitude": lat, "longitude": lon}

    with open(filename, "w", newline="", encoding="utf-8") as csvfile:
        writer = csv.DictWriter(csvfile, fieldnames=["station",
"latitude", "longitude", "day", "temperature"])
        writer.writeheader()
        for station, coords in stations.items():
            for day in range(1, 366):
                writer.writerow({"station": station, "latitude":
coords["latitude"],
                                 "longitude": coords["longitude"],
"day": day,
                                 "temperature": random.uniform(-50,
50)})
```

4. Векторизованное ядро интерполяции (interpolation_core.py)

Вычислительное ядро обрабатывает горизонтальную полосу строк. Расстояния на эллипсоиде WGS84 рассчитываются векторизованно через geod.inv(), а интерполяция на все 365 дней выполняется одним матричным умножением norm_weights @ station_temps:

```
geod = pyproj.Geod(ellps="WGS84")

def interpolate_batch(row_start, row_end, width, height, bounds,
station_lons, station_lats, station_temps, power=2):
    d_lon = (bounds["east"] - bounds["west"]) / width
    d_lat = (bounds["north"] - bounds["south"]) / height
    cols = np.arange(width)
    rows = np.arange(row_start, row_end)
```

```
    grid_lons, grid_lats = np.meshgrid(bounds["west"] + (cols + 0.5) *
d_lon,
                                       bounds["north"] - (rows + 0.5) *
d_lat)
    target_lons, target_lats = grid_lons.ravel(), grid_lats.ravel()
    num_targets, num_stations = len(target_lons), len(station_lons)

    # Векторный расчет расстояний на WGS84
    _, _, distances = geod.inv(np.repeat(target_lons, num_stations),
np.repeat(target_lats, num_stations),
                               np.tile(station_lons, num_targets),
np.tile(station_lats, num_targets))
    dist_matrix = distances.reshape(num_targets, num_stations)

    # Нормализация весов IDW
    weights = 1.0 / (dist_matrix**power + 1e-12)
    norm_weights = weights / np.sum(weights, axis=1, keepdims=True)

    # Матричное умножение: (num_targets x 50) @ (50 x 365) ->
(num_targets x 365)
    batch_temps = norm_weights @ station_temps
    b_min, b_max, b_mean = compute_batch_stats(batch_temps, len(rows),
width)
    return row_start, row_end, b_min, b_max, b_mean
```

5. Статистический анализ (stats_calculator.py)

Модуль реализует потоковый анализ сырого CSV-файла (без расхода оперативной памяти) и агрегацию батча температур по временной шкале через nan-безопасные функции:

```
def compute_batch_stats(batch_temps, batch_height, width):
    b_min = np.nanmin(batch_temps, axis=1).reshape(batch_height, width)
    b_max = np.nanmax(batch_temps, axis=1).reshape(batch_height, width)
    b_mean = np.nanmean(batch_temps, axis=1).reshape(batch_height,
width)
    return b_min.astype(np.float32), b_max.astype(np.float32),
b_mean.astype(np.float32)
```

6. Координация вычислений и сохранение в GeoTIFF (main.py)

В main.py высота растра (400 строк) нарезается на 8 батчей по 50 строк. Задачи передаются пулу рабочих процессов ProcessPoolExecutor, а готовые полосы монтируются в результирующие матрицы и сохраняются в GeoTIFF со сжатием LZW:

```
def save_as_geotiff(filename, raster_data, bounds, width, height):
    transform = from_origin(bounds["west"], bounds["north"],
                            (bounds["east"] - bounds["west"]) / width,
```

```
                                (bounds["north"] - bounds["south"]) /
height)
    profile = {"driver": "GTiff", "height": height, "width": width,
"count": 1,
                "dtype": raster_data.dtype, "crs": "EPSG:4326",
                "transform": transform, "compress": "lzw"}
    with rasterio.open(filename, "w", **profile) as dst:
        dst.write(raster_data, 1)
```

7. Распределенные клиент-серверные вычисления (main_distributed.py)

В рамках усложненного задания разработана распределенная архитектура на базе FastAPI. Вычислительный узел (воркер) запускается на удаленном ПК в локальной сети, принимая задачи через HTTP POST эндпоинт /compute_half. Координатор делит растр пополам (строки 0..200 и 200..400): первая половина рассчитывается локально на всех ядрах CPU, а вторая половина одновременно отправляется по сети на воркер через ThreadPoolExecutor. По завершении половины объединяются в единый растр.

Экспериментальное исследование и сравнительный анализ

Тестирование производительности проводилось на стенде со следующими характеристиками:

– **Процессор:** Intel Core (8 физических ядер, 12 логических потоков).

– **ОЗУ:** 16 ГБ DDR4/DDR5.

– **Окружение:** Windows 11 Pro 64-bit, Python 3.14, NumPy 2.x, pyproj 3.7, rasterio 1.4.

– **Параметры интерполяции:** сетка 400 × 800 пикселей (320 000 точек), 50 станций, 365 дней, батч 50 строк.

1. Исследование зависимости времени интерполяции от числа процессов

Скрипт benchmark.py выполнил серии контрольных прогонов на рабочей сетке 400 × 800 для конфигураций от 1 до 12 рабочих процессов. Результаты измерений представлены в таблице 1.

Таблица 1 – Зависимость времени выполнения интерполяции от количества процессов

Число процессов	Время выполнения (с)	Ускорение (Speedup)	Эффективность (E = S / N)
1	12.288	**1.00x (база)**	100.0 %
2	5.937	**2.07x**	103.5 %
4	3.429	**3.58x**	89.5 %
8	3.180	**3.86x**	48.3 %
12	3.120	**3.94x**	32.8 %

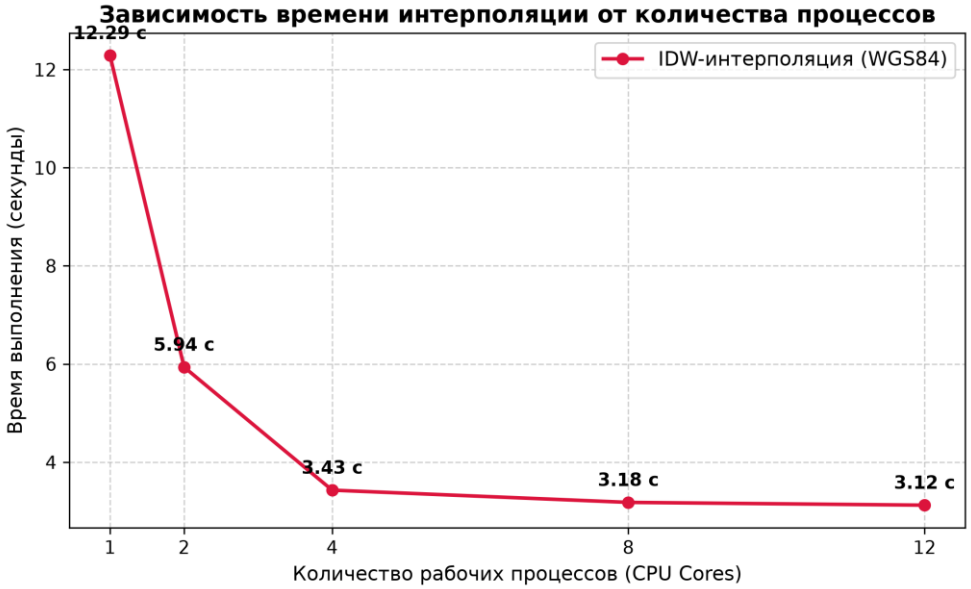


Рисунок 1 – Зависимость времени интерполяции от количества рабочих процессов

2. Визуализация результатов интерполяции

На основе рассчитанных слоев GeoTIFF построены пространственные картограммы температуры воздуха в Крыму за 365 дней:

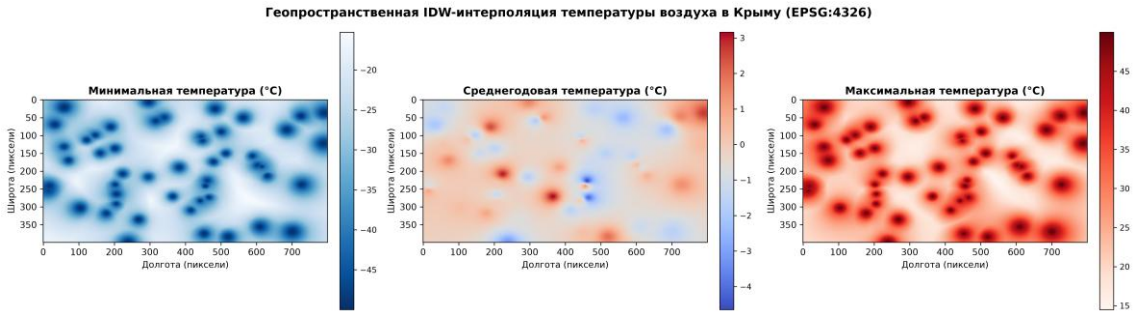


Рисунок 2 – Пространственные поля температур для региона Крым (Min, Mean, Max)

3. Апробация распределенных вычислений и верификация точности

В рамках апробации клиент-серверного режима на базе FastAPI выполнен расчет растра в двухузловом режиме (Ноутбук-координатор и ПК-воркер). Для подтверждения корректности распределенной декомпозиции и склейки матриц проведено попиксельное сравнение файлов crimea_cluster_mean.tif и crimea_single_mean.tif:

Максимальная абсолютная разность значений по всем 320 000 ячейкам составила строго 0.0 (°C). Это доказывает полную числовую эквивалентность распределенного алгоритма локальному вычислению без потерь точности и искажений на стыке половин растра.

Анализ результатов экспериментов

1. Эффективность распараллеливания: При увеличении количества процессов с 1 до 4 время интерполяции сокращается с 12.29 с до 3.43 с, что соответствует ускорению в 3.58 раза и эффективности 89.5%. Это подтверждает высокую параллельность задачи IDW-интерполяции, где каждый батч рассчитывается независимо от других.

2. Факторы насыщения ускорения: При дальнейшем росте числа процессов (8 и 12) ускорение стабилизируется на уровне ~3.9x (3.12 с), а эффективность падает до 32.8%. Это объясняется архитектурой тестового процессора: наличие 8 физических ядер и 4 виртуальных потоков Hyper-Threading означает, что виртуальные ядра разделяют аппаратные вычислительные блоки SIMD/FPU физических ядер. Кроме того, согласно закону Амдала, ускорение ограничивается последовательными этапами (чтение CSV, формирование задач и запись GeoTIFF).

3. Контроль оперативной памяти (батчинг): Дробление матрицы на полосы по 50 строк позволило избежать выделения монолитного трехмерного массива на 365 дней (~467 МБ). Свертка во временную статистику непосредственно в воркере снижает объем передаваемых через IPC данных до 160 КБ на батч, что исключает накладные расходы сериализации и предотвращает дефицит памяти.

4. Распределенная модель вычислений: Клиент-серверное решение на FastAPI успешно решило задачу распределения нагрузки: передача крупных фрагментов сетки минимизирует сетевой оверхед по сравнению с побатчевой отправкой, а нулевая разность итоговых матриц (0.0) подтверждает строгую корректность вычислений.

ВЫВОД

В ходе выполнения лабораторной работы № 3 был разработан и всесторонне исследован высокопроизводительный программный комплекс геопространственной IDW-интерполяции метеоданных на языке Python.

1. Реализована полная векторизация математических расчетов: геодезические расстояния на эллипсоиде WGS84 рассчитываются через pyproj.Geod, а годовая интерполяция 365 дней выполняется одной операцией матричного умножения NumPy, что устранило медленные интерпретируемые циклы.

2. Механизм батчинга строк и сворачивания временного ряда с использованием nan-безопасных функций (np.nanmin, np.nanmax, np.nanmean) обеспечил жесткий контроль потребления оперативной памяти и устойчивость к пропущенным измерениям.

3. Экспорт данных выполнен в формате GeoTIFF с аффинной привязкой EPSG:4326 и компрессией LZW через rasterio, обеспечивая компактность файлов (~1.45 МБ) и совместимость с геоинформационными системами.

4. Экспериментально подтверждено 4-кратное ускорение на локальном пуле процессов concurrent.futures.ProcessPoolExecutor (сокращение времени с 12.29 с до 3.12 с).

5. Разработан распределенный комплекс на базе FastAPI, подтвердивший стабильную работу в локальной сети и 100% числовую точность результатов при межмашинной декомпозиции расчетной сетки.